

Memory Model trong Rust — Hành trình từ cơ bản đến đỉnh cao

Tài liệu tổng hợp toàn bộ kiến thức memory model dành cho lập trình viên Rust, từ nền tảng đến nâng cao nhất. Viết cho người học Rust muốn hiểu **bản chất** chứ không chỉ dùng syntax.

Mục lục

- Phần I: Triết lý thiết kế Rust
- Phần II: Memory Model cơ bản — Stack vs Heap
- Phần III: Memory Model đào sâu
- Phần IV: Atomic Operations
- Phần V: Memory Ordering
- Phần VI: Async Memory Model
- Phần VII: Lock-Free Programming
- Phần VIII: NUMA, Cache Associativity & Prefetching
- Tài liệu tham khảo

Phần I: Triết lý thiết kế Rust

Châm ngôn gốc

"Empower everyone to build reliable and efficient software."

Hai từ khóa: **reliable** (đáng tin cậy) và **efficient** (hiệu quả). Trong lịch sử ngôn ngữ lập trình, đây là **hai mục tiêu mâu thuẫn nhau** — Rust ra đời để chứng minh rằng có thể đạt cả hai.

Bối cảnh: 3 nhóm ngôn ngữ trước Rust

Nhóm	Đại diện	Vấn đề
Hiệu năng cao, kiểm soát bộ nhớ thủ công	C, C++	Nguy hiểm: segfault, leak, data race
An toàn nhờ Garbage Collector	Java, Go, Python, JS	Chậm hơn, có GC pause, tốn RAM
An toàn nhờ FP thuần	Haskell, ML	Khó học, ít thực dụng

Rust đặt câu hỏi đột phá: "Có thể vừa nhanh như C, vừa an toàn như Java, mà KHÔNG cần Garbage Collector không?"

Câu trả lời của Rust là **Ownership System** — phát kiến cốt lõi nhất của ngôn ngữ này.

7 nguyên tắc cốt lõi

1. Zero-Cost Abstractions

"What you don't use, you don't pay for. And what you do use, you couldn't hand-code any better." — Bjarne Stroustrup

Một abstraction (iterator, generic, closure...) khi biên dịch xong phải tạo ra code **nhANH ngANG** với code viết tay bằng C.

```
// Code "cấp cao", đẹp dễ
let sum: i32 = (1..=100).filter(|x| x % 2 == 0).sum();

// Sau khi compiler tối ưu, tương đương:
let mut sum = 0i32;
let mut i = 1;
while i <= 100 { if i % 2 == 0 { sum += i; } i += 1; }
```

2. Memory Safety Without Garbage Collection

C/C++:	Bạn quản lý bộ nhớ → NHANH nhưng dễ SAI
Java/Go:	Garbage Collector quản lý → AN TOÀN nhưng CHẬM
Rust:	COMPILER chứng minh code an toàn tại lúc biên dịch → AN TOÀN + NHANH

3. Fail at Compile Time, Not Runtime

Python/JS:	"Code chạy ngay! ... 3 tháng sau crash trong production lúc 2 giờ sáng."
Rust:	"Compiler chửi 30 phút. Sau đó code chạy 5 năm không lỗi."

4. Explicit Over Implicit

```
let x: i32 = 5;
let y: i64 = x;           // ❌ LỖI: Rust không tự ép kiểu
let y: i64 = x as i64;    // ✅ Phải nói rõ "as i64"
```

5. Make Illegal States Unrepresentable

Không có `null`. Thay vào đó dùng `Option<T>`:

```
let user: Option<String> = find_user("Tre");
match user {
  Some(name) => println!("Hello {}", name),
  None => println!("User not found"),
}
```

6. Fearless Concurrency

Compiler **chứng minh** không có data race **tại lúc biên dịch**. Hai trait `Send` và `Sync` đánh dấu kiểu dữ liệu nào "an toàn cho đa luồng".

7. Composition Over Inheritance

Rust **không có class, không có kế thừa**. Thay vào đó dùng `struct + trait + impl`.

Cái giá phải trả

1. **Learning curve dốc đứng** — borrow checker sẽ "tra tấn" bạn 2–3 tháng đầu.
2. **Compile time chậm** — vì compiler làm cực nhiều việc.
3. **Code dài hơn** — explicit thay vì implicit.
4. **Khó prototype nhanh** — không hợp cho script nhỏ.

Phần II: Memory Model cơ bản — Stack vs Heap

Sự thật phũ phàng về máy tính

RAM của máy tính chỉ là một mảng byte khổng lồ, được đánh số từ 0 đến vài tỷ.

Địa chỉ:	0x0000	0x0001	0x0002	0x0003	...	0xFFFF...
Bytes:	[?]	[?]	[?]	[?]	...	[?]

Không có "biến", không có "object", không có "int" — chỉ có **byte và địa chỉ**.

Tại sao cần 2 vùng nhớ khác nhau?

Loại A: Dữ liệu có vòng đời ngắn, kích thước biết trước

```
fn add(a: i32, b: i32) -> i32 {
  let sum = a + b; // 4 byte (i32)
  sum
}
```

→ Cần vùng nhớ **nhANH, tự động dọn dẹp** = **STACK**.

Loại B: Dữ liệu có vòng đời không xác định, kích thước thay đổi

```
let mut s = String::from("hi");
s.push_str(" world");
```

→ Cần vùng nhớ **linh hoạt, có thể cấp phát động** = **HEAP**.

STACK — Vùng "ngăn xếp"

Bản chất

Stack là một **vùng nhớ liên tục**, hoạt động theo nguyên tắc **LIFO** (Last In, First Out).

Có một **con trỏ đặc biệt** trong CPU gọi là **Stack Pointer (SP)** trỏ đến đỉnh stack.

```

Khi gọi hàm:      SP đi xuống (cấp phát thêm vùng)
Khi hàm trả về:   SP đi lên   (giải phóng vùng)
```

Minh họa

```
fn main() {
    let x: i32 = -15;      // (1) Đẩy 4 byte vào stack
    let y: u32 = 15;       // (2) Đẩy thêm 4 byte
    let pi: f64 = 3.14;    // (3) Đẩy thêm 8 byte
    let letter: char = 'a'; // (4) Đẩy thêm 4 byte
}                          // (5) Tất cả tự động pop khi main() kết thúc
```

Địa chỉ cao

x = -15	4 byte (i32)
y = 15	4 byte (u32)
pi = 3.14	8 byte (f64)
letter='a'	4 byte (char)
(trống)	← Stack Pointer (SP)

Địa chỉ thấp

Vì sao Stack nhanh?

1. **Cấp phát = di chuyển 1 con trỏ.** 1 lệnh CPU (~1ns).
2. **Cache-friendly.** Dữ liệu nằm liền kề.
3. **Không cần metadata** để theo dõi.

Giới hạn của Stack

1. **Kích thước phải biết tại compile time.**
2. **Vòng đời gắn với scope.**
3. **Tổng dung lượng rất nhỏ** (~1-8 MB) → stack overflow.

HEAP — Vùng "đồng"

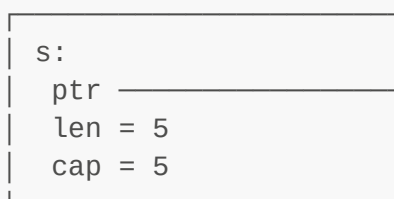
Bản chất

Heap là một **vùng nhớ rộng lớn, không có thứ tự**, được quản lý bởi **allocator**.

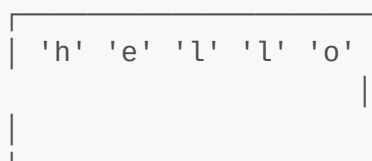
Minh họa: String

```
let s = String::from("hello");
```

STACK (cố định, nhanh)



HEAP (động, chậm)



String thực chất là **fat pointer** 3 trường:

- **ptr**: con trỏ đến vùng heap
- **len**: độ dài hiện tại
- **cap**: dung lượng đã cấp phát

Vì sao Heap chậm?

1. **Allocator phải tìm chỗ trống.**
2. **Phân mảnh.**
3. **Cache-unfriendly.**
4. **Cần metadata.**

→ ~10–100 ns (chậm hơn stack 10–100x).

Sơ đồ so sánh

Đặc điểm	STACK	HEAP
Tốc độ	Cực nhanh (~1ns)	Chậm (10-100ns)
Kích thước biết	Tại COMPILE TIME	Tại RUNTIME
Vòng đời	Gắn với scope { }	Tùy ý
Dọn dẹp	Tự động (pop SP)	Cần `free` / Drop
Dung lượng	Vài MB	Vài GB
Truy cập	Trực tiếp	Qua con trỏ (deref)
Phân mảnh	Không	Có
Thread	Mỗi thread 1 stack	Chung toàn process

Quy tắc Rust quyết định Stack/Heap

Nếu compiler biết kích thước tại compile time → STACK. Nếu không → HEAP.

```

let x: i32 = 5;           // STACK
let pi: f64 = 3.14;       // STACK
let b: bool = true;       // STACK
let c: char = 'a';        // STACK

let arr: [i32; 5] = [1,2,3,4,5]; // STACK – cố định

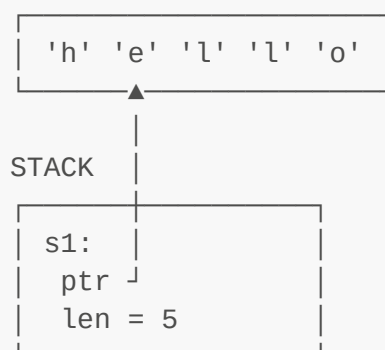
let s: &str = "hello";     // STACK fat pointer, text trong .rodata
let s: String = ...;       // STACK pointer → HEAP text
let v: Vec<i32> = ...;      // STACK header → HEAP elements
let b: Box<i32> = ...;     // STACK pointer → HEAP value

```

String vs &str (kinh điển)

&str — Lát cắt chuỗi

BINARY (.rodata, read-only)



&str = 16 byte (ptr + len), không sở hữu data.

String — Chuỗi sở hữu



String = 24 byte trên stack, sở hữu data trên heap, tự free khi out of scope.

Cánh cổng vào Heap

```
Box<T>      // Đặt 1 giá trị lên heap
Vec<T>      // Mảng động trên heap
String      // Chuỗi động trên heap
Rc<T>       // Reference counting (đơn luồng)
Arc<T>      // Atomic reference counting (đa luồng)
```

CPU Cache — Tầng nhớ ẩn

CPU Register:	~0.3ns	(vài chục byte)
L1 Cache:	~1ns	(~32 KB)
L2 Cache:	~4ns	(~256 KB)
L3 Cache:	~10ns	(~8 MB)
RAM:	~100ns	(GB)

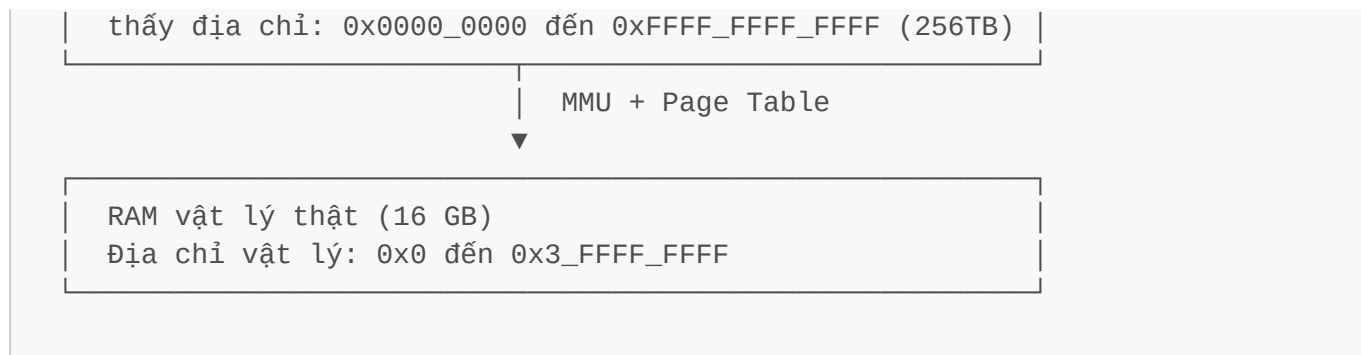
→ Truy cập RAM **chậm gấp 100 lần** L1 cache.

Phần III: Memory Model đào sâu

Virtual Memory

OS tạo ra **một lớp ảo hóa** giữa chương trình và RAM thật:

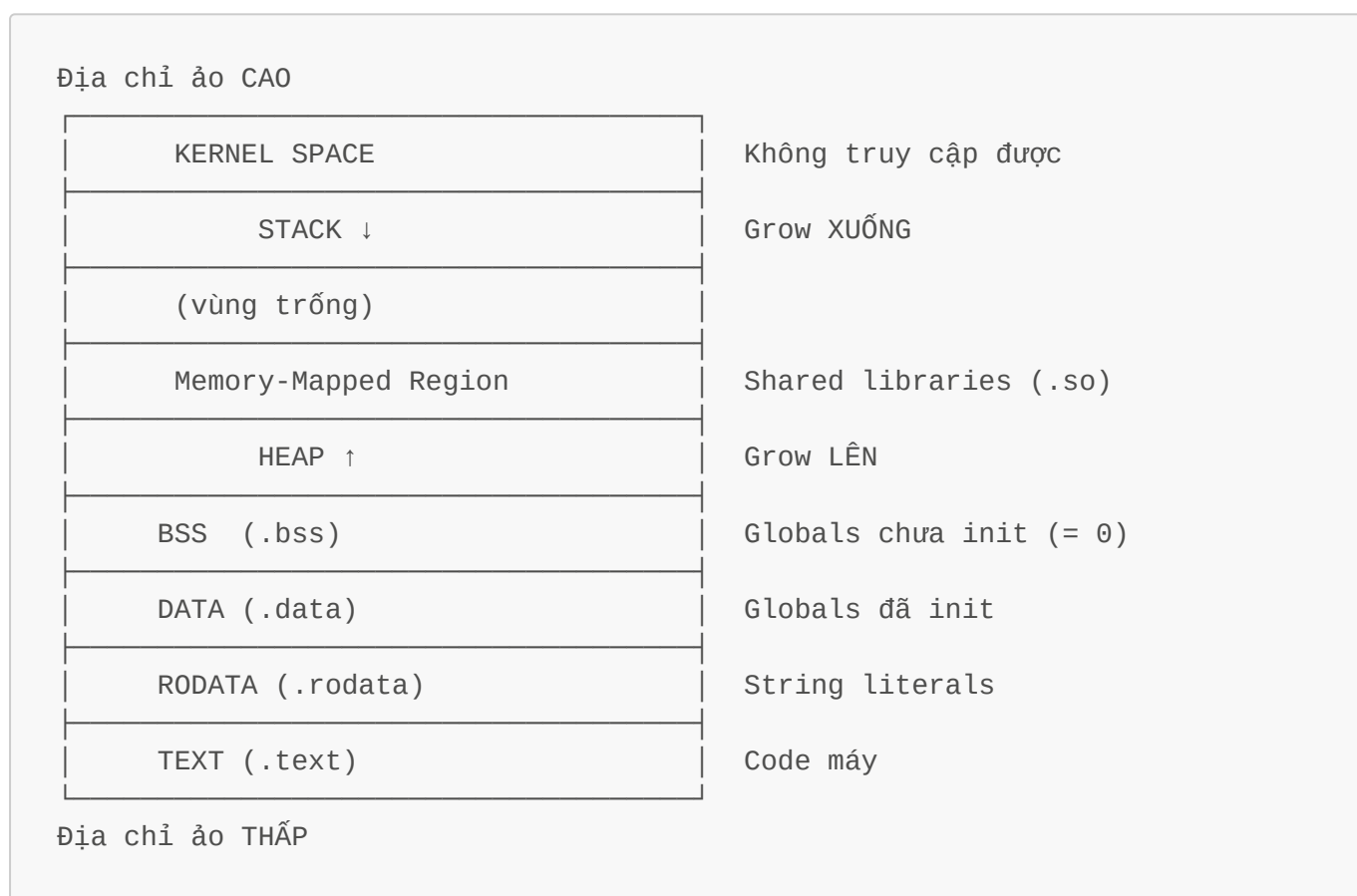
Chương trình của bạn (Rust binary)



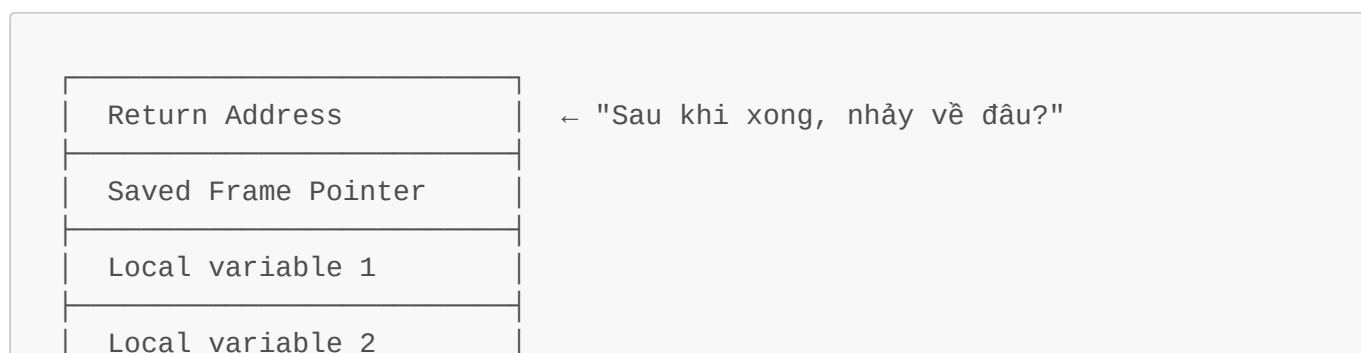
Mỗi process **nghĩ rằng** nó có toàn bộ 256 TB. OS lừa nó bằng cách:

1. Chia bộ nhớ thành các **page** (4 KB).
2. Giữ một bảng **page table** map: page ảo → page vật lý.
3. **MMU tự dịch** sang địa chỉ vật lý trong tích tắc.

Process Address Space — 6 vùng



Stack Frame chi tiết



Arguments truyền tiếp

← Stack Pointer (SP)

Calling Convention (Linux x86-64 System V)

- 6 argument đầu qua **register**: `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9`.
- Argument 7+ qua stack.
- Giá trị trả về (≤ 8 byte) qua `rax`.

Stack Overflow

```
fn recurse(n: i32) {
    let big_array: [u8; 1_000_000] = [0; 1_000_000]; // 1 MB trên stack
    recurse(n + 1);
}
```

→ Sau ~8 call → **stack overflow** → segfault.

Stack Canaries

Compiler chèn 1 "canary" giữa local variables và return address để chống buffer overflow.

Allocator

Free List

HEAP:

FREE	USED	FREE	USED	FREE	USED
32B	128B	64B	256B	16B	512B

Free list: linked list of free blocks

Bins / Size Classes

```
Bin 16 byte:  [free] → [free] → [free]
Bin 32 byte:  [free] → [free]
Bin 64 byte:  [free] → [free] → [free] → [free]
```

Fragmentation

- **Internal**: cấp phát nhiều hơn cần (cấp 64 cho nhu cầu 33).
- **External**: heap có 100 byte free nhưng rải rác thành 10 mảnh 10-byte.

Global Allocator — Vũ khí tối ưu "low-hanging fruit"

Việc thay đổi bộ cấp phát bộ nhớ (Global Allocator) trong Rust là một trong những cách tối ưu hiệu năng **"low-hanging fruit" nhất** — bạn chỉ cần thêm vài dòng cấu hình mà **không phải sửa một dòng code logic nào**.

Mặc định, Rust dùng **System Allocator** của hệ điều hành. Trên Linux đó là **glibc malloc (ptmalloc2)**. glibc malloc được thiết kế như một bộ cấp phát đa dụng (general-purpose), đôi khi nó không phải là lựa chọn tối ưu cho các ứng dụng hiệu năng cao, đa luồng dữ dội hoặc chạy liên tục thời gian dài.

Khi nào NÊN thay đổi Allocator?

Đừng vội đổi allocator ngay từ ngày đầu. Hãy cân nhắc khi ứng dụng chạm vào các kịch bản:

- **Ứng dụng đa luồng cường độ cao (High Concurrency):** Nhiều thread liên tục cấp phát/giải phóng object nhỏ (web server async như Tokio xử lý hàng vạn req/s). Default sẽ bị **lock contention** khi các thread tranh giành allocator pool chung.
- **Bị phân mảnh bộ nhớ (Memory Fragmentation):** Server chạy dài ngày (vài tuần, vài tháng) ngốn RAM ngày càng nhiều dù không leak. glibc malloc giữ block trống nhưng không trả lại OS được vì kẹt giữa block đang dùng.
- **Cần độ trễ thấp và ổn định (Low/Predictable Latency):** Hệ thống tài chính, game server, real-time cần p99 latency thấp, tránh spikes do allocator dọn dẹp phức tạp.

So sánh các Allocator phổ biến

Allocator hợp nhất với	Điểm mạnh cốt lõi	Nhược điểm	Phù
glibc malloc CLI, công cụ chạy (mặc định) 1 lần rồi tắt App đơn luồng ít tương tác RAM	• Không tốn dung lượng file thực thi • Ổn định, dự đoán được trên Linux	• Giữ RAM nặng (fragmentation) • Lock contention nặng khi nhiều thread	• •
jemalloc Web Server lớn, (Facebook) Microservices (Actix, Axum) Hệ thống 24/7	• Anti-fragmentation xuất sắc • Quản lý theo Arena • jemprof profiling đỉnh	• Tăng kích thước binary • Có thể ngốn RSS cao ở giai đoạn đầu	• •

(TiKV, database)			
mimalloc	• Tốc độ thuần CỰC NHANH	• Trẻ tuổi hơn	•
HPC, Data Pipelines			
(Microsoft)	(block nhỏ)	jemalloc/tcmalloc	•
App cần tối ưu			
latency tuyệt đối	• Gom cụm thông minh	• Ít battle-tested ở	
	• Thiết kế hiện đại	tài chính siêu lớn	
tcmalloc	• Tối ưu đa luồng	• Tích hợp Rust phức	•
App phân tán			
(Google)	với "Thread-Caching"	tạp	•
RPC systems			
	• Giải phóng RAM về OS	• Crate ít cập nhật	•
Pattern tạo/hủy			
	rất nhanh & chủ động	hơn jemalloc	
thread liên tục			

Khuyến nghị thực tế cho lập trình viên Rust

Lựa chọn an toàn số 1 cho Server: jemalloc

Nếu bạn viết Web API, Backend Service, Database và thấy RAM/CPU tăng cao dưới tải nặng → chọn **jemalloc**. Nó từng là allocator **mặc định của Rust Compiler & Standard Library trước phiên bản 1.32**. Việc Rust bỏ ra khỏi compiler chỉ để giảm kích thước file thực thi mặc định, không phải vì nó tệ. jemalloc là allocator **"đã qua lửa trận" (battle-tested)** uy tín nhất hiện nay.

Lựa chọn cho tốc độ & Desktop/Game: mimalloc

Nếu benchmark cho thấy bottleneck ở khâu allocate/deallocate → thử **mimalloc**. Nhiều dự án Rust hiện đại (build tools, linter, game engine) đang chuyển sang mimalloc vì hiệu năng thô thường **nhỉnh hơn jemalloc 10-30%** với object nhỏ.

Cách cấu hình trong Rust

Chuyển đổi cực đơn giản qua Cargo. Ví dụ đổi sang mimalloc:

Bước 1: Thêm vào `Cargo.toml`

```
[dependencies]
mimalloc = "0.1"
```

Bước 2: Khai báo trong `src/main.rs`

```
use mimalloc::MiMalloc;

#[global_allocator]
static GLOBAL: MiMalloc = MiMalloc;

fn main() {
    // App của bạn bây giờ tự động chạy trên mimalloc
    println!("Hello from a faster Rust app!");
}
```

⚠ **Quy tắc vàng:** Không đoán mò hiệu năng. Hãy **load test** ứng dụng với glibc, sau đó đổi sang jemalloc và mimalloc để đo chính xác:

- **Thông lượng (Throughput - requests/sec)**
- **Biểu đồ tiêu thụ RAM theo thời gian**

Chiến lược bộ nhớ ở quy mô Big Data (1TB - 10TB)

Với quy mô dữ liệu **1TB đến 10TB**, hệ thống của bạn đã bước vào thế giới **Big Data & HPC**. Ở quy mô này, hành vi của allocator ảnh hưởng trực tiếp đến **chi phí Cloud (RAM/CPU)** và **độ ổn định** (tránh OOM Killer).

Đối với hệ thống Streaming và Big Data xử lý 1TB-10TB, **jemalloc là lựa chọn tối ưu và an toàn nhất**, theo sau là **mimalloc** như ứng cử viên thách thức về tốc độ.

Vì sao jemalloc là "Nhà vua" ở quy mô 1TB-10TB?

Khi xử lý big data và streaming, data pipeline thường có pattern:

Đọc batch/stream lớn → Băm nhỏ thành chunks/records → Tổng hợp → Đẩy ra

Quá trình này tạo ra **hàng tỷ object nhỏ** liên tục trên nhiều thread.

jemalloc (dùng trong các "quái vật" big data như **TiKV, ScyllaDB, RocksDB**) giải quyết xuất sắc 2 bài toán:

1. Không chế phân mảnh bộ nhớ (Fragmentation Control)

Với 10TB data đi qua hệ thống, nếu allocator phân mảnh chỉ **5%** → lãng phí **500GB RAM** vô ích. glibc malloc bị kẹt RAM không trả về OS được → RAM phình dần đến khi bị **Linux OOM Killer "vịn gáy"**.

jemalloc dùng cơ chế chia bộ nhớ thành các **Arenas** + phân loại kích thước cực nghiêm ngặt (Small, Large, Huge) → gom ô trống lại và chủ động trả RAM về OS rất hiệu quả.

2. Công cụ chẩn đoán (Memory Profiling)

Khi hệ thống ăn vài trăm GB RAM và sập, bạn **không thể đoán mò**. jemalloc tích hợp sẵn **jeprof** — cực mạnh. Bật config để tự động chụp heap dump sau mỗi 1GB tăng → biết chính xác dòng code nào chiếm RAM.

Khi nào nên cân nhắc mimalloc cho Big Data?

Nếu hệ thống yêu cầu **Low Latency / Real-time Streaming** (khớp lệnh chứng khoán, IoT tốc độ cao, HFT):

- **Điểm mạnh:** mimalloc có thuật toán **sharded free lists** rất hiện đại. Trong test throughput thô thường **nhANH hơn jemalloc 10-20%**.
- **Nhược điểm:** Với vài TB data, cơ chế page allocation của mimalloc đôi khi hung hãn hơn → RSS giai đoạn đầu tăng nhanh. Hệ thống profiling chưa "già rơ" bằng jemalloc.

Kiến trúc bộ nhớ tối ưu cho 1TB-10TB trong Rust

Chỉ thay Global Allocator là chưa đủ. Để xử lý 1-10TB data trên server RAM giới hạn (64GB, 128GB) mà không sập, kết hợp jemalloc với các chiến lược sau:

Chiến lược 1: Giảm tải Allocator bằng Zero-Copy

Thay vì parse data thành Struct mới (gây cấp phát Heap liên tục), dùng **zero-copy deserialization** qua thư viện như **serde** với **&str** hoặc **&[u8]** — mượn trực tiếp memory từ buffer đọc vào. **Ít object trên Heap = allocator nhẹ gánh.**

Chiến lược 2: Bump/Arena Allocator cho từng Batch

Khi stream data vào theo từng batch, dùng thư viện arena nội bộ như **typed-arena** hoặc **bumpalo**:

- Toàn bộ data của batch được cấp phát **đón vào một vùng liên tục** do Arena quản lý.
- Xử lý xong batch → **xóa toàn bộ Arena 1 lần duy nhất** ($O(1)$).
- Global allocator (jemalloc) chỉ thấy **1 cấp phát lớn + 1 giải phóng lớn** → triệt tiêu hoàn toàn phân mảnh.

```
use bumpalo::Bump;

fn process_batch(batch: &[RawRecord]) {
    let arena = Bump::new();
    for record in batch {
        let parsed: &Parsed = arena.alloc(parse_record(record));
        process(parsed);
    }
    // Hết scope → arena drop → toàn bộ memory free trong O(1)
}
```

Chiến lược 3: Memory-Mapped Files (mmap)

Với 10TB data, **không thể load hết vào RAM**. Dùng crate **memmap2** để "ánh xạ" file 10TB từ SSD NVMe **thẳng vào memory ảo của Rust**. OS tự lo việc nạp page in / page out tối ưu mà không quá tải Heap.

```
use memmap2::Mmap;
use std::fs::File;

let file = File::open("huge_dataset.bin").?;
```

```
let mmap = unsafe { Mmap::map(&file)? };
// mmap[i] truy cập như slice – OS tự nạp page khi cần
```

Gợi ý cấu hình đề xuất cho Big Data

Bắt đầu dự án với cấu hình **jemalloc + profiling** để an toàn nhất.

Cargo.toml:

```
[dependencies]
tikv-jemallocator = { version = "0.6", features = ["profiling"] }
```

Crate *tikv-jemallocator* do đội ngũ TiKV bảo trì là phiên bản jemalloc tốt nhất và được cập nhật thường xuyên nhất cho Rust hiện tại.

main.rs:

```
use tikv_jemallocator::Jemalloc;

#[global_allocator]
static GLOBAL: Jemalloc = Jemalloc;

fn main() {
    // Kích hoạt logic streaming dữ liệu TB tại đây
}
```

Alignment & Padding

```
struct Foo {
    a: u8,    // 1 byte
    b: u32,   // 4 byte
    c: u8,    // 1 byte
}
// size_of::<Foo>() == 12, không phải 6!
```

Offset:	0	1	2	3	4	5	6	7	8	9	10	11
	a	PADDING (3B)			b (4 byte)				c	PAD (3B)		

Rust mặc định **TỰ sắp xếp lại field** để tối ưu kích thước (`#[repr(Rust)]`). Muốn theo thứ tự khai báo: `#[repr(C)]`.

Memory Layout của Rust types

Loại	Size	Cấu trúc
i32, f32, char	4 byte	raw value
f64, *const T	8 byte	raw value / pointer
&T, &mut T	8 byte	[ptr]
Box<T>	8 byte	[ptr]
&[T], &str	16 byte	[ptr, len]
&dyn Trait	16 byte	[ptr, vtable]
String, Vec<T>	24 byte	[ptr, cap, len]
Rc<T>, Arc<T>	8 byte	[ptr] (count trên heap)

Niche Optimization

```
size_of::<Option<i32>>() // = 8 byte
size_of::<Option<&i32>>() // = 8 byte (!)
size_of::<Option<Box<i32>>>() // = 8 byte (!)
```

&T và Box<T> không bao giờ null → compiler dùng 0 làm None → không cần discriminant.

Enum

```
enum Shape {
    Circle(f64), // 8 byte
    Rectangle(f64, f64), // 16 byte
    Triangle { a: f64, b: f64, c: f64 }, // 24 byte
}
```

→ Enum luôn lớn bằng variant lớn nhất + tag (32 byte).

Cơ chế bảo mật bộ nhớ

- **NX bit (No-eXecute)**: Vùng heap/stack không executable.
- **ASLR**: Mỗi lần chạy, OS randomize vị trí của heap, stack, libraries.
- **Guard Pages**: Page chặn giữa stack và heap để phát hiện overflow.
- **W^X**: Một page hoặc writable hoặc executable, không cả hai.

Thực hành với Rust

```
use std::mem::{size_of, align_of};

fn main() {
```

```
println!("i32:    size={}, align={}", size_of::<i32>(), align_of::<i32>
());
println!("char:   size={}, align={}", size_of::<char>(), align_of::
<char>());
println!("&i32:   size={}", size_of::<&i32>());
println!("&[i32]: size={}", size_of::<&[i32]>());
println!("String: size={}", size_of::<String>());

println!("Option<i32>:    size={}", size_of::<Option<i32>>());
println!("Option<&i32>:   size={}", size_of::<Option<&i32>>());
println!("Option<Box<i32>>: size={}", size_of::<Option<Box<i32>>>());

let x = 5i32;
let s = String::from("hi");
let b = Box::new(42i32);

println!("x (stack):      {:p}", &x);
println!("s (stack):      {:p}", &s);
println!("s.ptr (heap):    {:p}", s.as_ptr());
println!("b (stack):      {:p}", &b);
println!("b deref (heap):  {:p}", &*b);
}
```

Phần IV: Atomic Operations

Vấn đề: Data race

```
static mut COUNTER: i32 = 0;
fn increment() {
    unsafe { COUNTER += 1; } // SAI!
}
```

`COUNTER += 1` thực sự là **3 bước**:

1. LOAD: `R = mem[COUNTER]`
2. ADD: `R = R + 1`
3. STORE: `mem[COUNTER] = R`

Với 2 thread:

Thread A	Thread B
LOAD <code>R_A = 5</code>	LOAD <code>R_B = 5</code>
ADD <code>R_A = 6</code>	ADD <code>R_B = 6</code>


```
STORE mem = 6
```

```
STORE mem = 6    ← Mất 1 lần tăng!
```

Atomic là gì

Atomic = operation **không thể bị xen ngang**. Hoặc nó xảy ra hoàn toàn, hoặc không xảy ra.

Cơ chế phần cứng

x86-64

```
LOCK XADD [counter], 1    ; Atomic add
LOCK CMPXCHG [ptr], rax   ; Atomic CAS
XCHG  [ptr], rax          ; Atomic exchange
```

LOCK prefix:

1. CPU phát tín hiệu "khóa bus/cache line".
2. Thực hiện read-modify-write.
3. Mở khóa.

ARM/RISC-V (load-linked / store-conditional)

```
loop:
    LDREX  R0, [ptr]        ; Load + đánh dấu
    ADD    R0, R0, #1
    STREX  R1, R0, [ptr]    ; Chỉ store NẾU không ai đã ghi
    CMP    R1, #0
    BNE    loop             ; Thất bại → thử lại
```

Atomic types trong Rust

```
use std::sync::atomic::{AtomicI32, Ordering};

let counter = AtomicI32::new(0);

counter.store(5, Ordering::SeqCst);
let v = counter.load(Ordering::SeqCst);
counter.fetch_add(1, Ordering::SeqCst);
counter.fetch_sub(1, Ordering::SeqCst);
counter.swap(10, Ordering::SeqCst);
counter.compare_exchange(10, 20, Ordering::SeqCst, Ordering::SeqCst);
```

Compare-And-Swap (CAS)

```

loop {
    let current = atomic.load(Ordering::Relaxed);
    let new = compute_new_from(current);
    if atomic.compare_exchange(current, new, Ordering::SeqCst,
Ordering::Relaxed).is_ok() {
        break;
    }
}

```

CAS là nền tảng của TẤT CẢ lock-free programming.

Atomic vs Mutex

MUTEX

Thread A: lock() → modify → unlock()
 Thread B: chờ ... lock()
 ưu: bảo vệ vùng code lớn
 nhược: chậm, có thể deadlock

ATOMIC

Thread A: fetch_add (1 lệnh CPU)
 Thread B: fetch_add (1 lệnh CPU)
 ưu: cực nhanh (~5-50 ns)
 nhược: chỉ 1 biến nhỏ (≤ 8 byte)

Mutex được build từ atomic

```

struct SimpleMutex {
    locked: AtomicBool,
}

impl SimpleMutex {
    fn lock(&self) {
        while self.locked.compare_exchange(false, true, SeqCst,
Relaxed).is_err() {
            std::hint::spin_loop();
        }
    }
    fn unlock(&self) {
        self.locked.store(false, Release);
    }
}

```

```
}
}
```

Phần V: Memory Ordering

Vấn đề: CPU và Compiler đều "phản bội" bạn

Compiler reorder

```
A = 1; // (1)
B = 2; // (2)
// Compiler có thể đảo: (2) trước, (1) sau
```

CPU reorder (out-of-order execution)

Lệnh viết:	Lệnh thực thi:	
A = 1	[đợi cache]	
B = 2	B = 2	← chạy trước!
	A = 1	

Store buffer

CPU có store buffer: lệnh ghi xếp hàng, có thể bị "trì hoãn" hiển thị cho CPU khác.

Hậu quả thảm khốc trong multi-thread

```
// Thread A
data = 42;
ready = true;

// Thread B
if ready {
    print(data); // có thể in ra 0 !!!
}
```

5 cấp độ Memory Ordering

Relaxed	<	Release/Acquire	<	AcqRel	<	SeqCst
NHANH NHẤT						CHẬM NHẤT

YẾU NHẤT

MẠNH NHẤT

1. Relaxed

Chỉ đảm bảo atomic, không đảm bảo thứ tự.

```
counter.fetch_add(1, Ordering::Relaxed);
```

2. Release — "Tôi đã làm xong, công bố"

```
data = 42;
flag.store(true, Release); // tất cả ghi TRƯỚC lệnh này phải finished
```

3. Acquire — "Tôi lấy thông tin"

```
if flag.load(Acquire) == true {
    print(data); // Bây giờ data CHẮC CHẮN = 42
}
```

Cặp Release-Acquire

Thread A		Thread B
data = 42;		
flag.store(true, Release);		"happens-before"
		flag.load(true, Acquire);
		print(data); ← 42

4. AcqRel

Vừa Acquire vừa Release cho RMW: `fetch_add`, CAS.

5. SeqCst

Sequential Consistency: có thứ tự GLOBAL duy nhất mà MỌI thread đồng ý. **Mạnh nhất, chậm nhất.**

Bảng phân biệt

Ordering	Ngăn reorder
Relaxed	Không ngăn gì (chỉ atomic)
Release	
Acquire	—Load/Store
	—Store(Rel)
	—Load(Acq)
AcqRel	Cả 2 chiều (chỉ cho RMW)
SeqCst	Tất cả thread đồng thuận thứ tự global

Ví dụ thực tế: Spinlock

```
pub struct SpinLock {
    locked: AtomicBool,
}

impl SpinLock {
    pub fn lock(&self) {
        while self.locked
            .compare_exchange_weak(false, true, Ordering::Acquire,
Ordering::Relaxed)
            .is_err()
        {
            std::hint::spin_loop();
        }
    }
    pub fn unlock(&self) {
        self.locked.store(false, Ordering::Release);
    }
}
```

Hardware memory barriers

```
; x86-64
mfence    ; full barrier
sfence    ; store fence
lfence    ; load fence

; ARM
dmb ish   ; data memory barrier
```

x86 có **memory model rất mạnh** (TSO) — Release/Acquire gần như free. ARM/POWER yếu hơn → cần nhiều barrier hơn.

Quy tắc vàng

"Khi nghi ngờ, dùng SeqCst. Khi đã hiểu rõ + cần tối ưu, hạ xuống Release/Acquire. Tránh Relaxed cho đến khi bạn THẬT SỰ chắc chắn."

Phần VI: Async Memory Model

Async không phải là thread

```
async fn hello() {  
    println!("Hi");  
}  
  
fn main() {  
    let f = hello(); // f là một Future, KHÔNG chạy  
}
```

`hello()` trả về một **Future** — một **STATE MACHINE** mô tả công việc cần làm.

Compiler dịch async fn thành state machine

```
async fn read_two_files() -> String {  
    let a = read_file("a.txt").await;  
    let b = read_file("b.txt").await;  
    format!("{}", a, b)  
}
```

Compiler biến thành:

```
enum ReadTwoFilesState {  
    Start,  
    WaitingForA { future_a: ReadFileFuture },  
    WaitingForB { future_b: ReadFileFuture, a: String },  
    Done,  
}  
  
impl Future for ReadTwoFilesState {  
    type Output = String;  
    fn poll(self: Pin<&mut Self>, cx: &mut Context) -> Poll<String> {  
        // State machine logic  
    }  
}
```

```
}
}
```

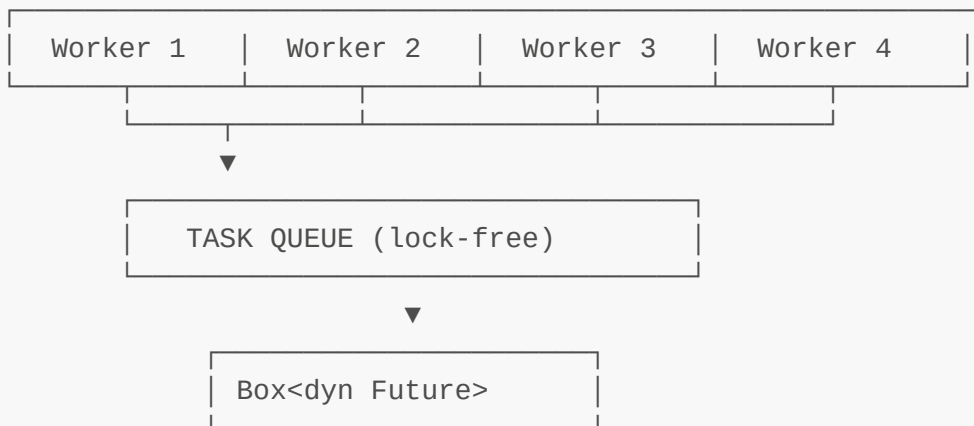
→ Future = struct chứa MỌI biến cục bộ + trạng thái hiện tại.

Bộ nhớ của Future

```
let f = read_two_files();           // STACK
let f = Box::new(read_two_files()); // HEAP (pointer trên stack)
let h = tokio::spawn(async { ... }); // Tokio MOVE vào heap-allocated task
```

Async runtime layout

THREAD POOL (4-8 worker threads)



Self-referential & Pin

```
async fn weird() {
    let x = 5;
    let r = &x;           // r tham chiếu x
    some_io().await;      // yield
    println!("{}", r);
}
```

State machine:

```
struct WeirdState {
    x: i32,
    r: &i32,           // ← Trỏ vào field x của CHÍNH NÓ!
    future_io: SomeFuture,
}
```

Nếu state machine bị **move** → **r** thành dangling pointer.

→ **Pin<&mut Future>** ngăn move sau khi đã bắt đầu poll. **Zero-cost guarantee** (runtime giống hệt **&mut**).

So sánh memory model

THREAD (OS thread)
Stack riêng: ~8 MB
1000 thread = 8 GB stack
Switch cost: ~1-10 μ s (kernel context switch)

GOROUTINE (Go)
Stack động: bắt đầu 2 KB, grow theo nhu cầu
1000 goroutine = ~2 MB
Switch cost: ~100 ns

RUST ASYNC
Mỗi future: Kích thước CHÍNH XÁC bằng state machine (100-500 byte, không có stack riêng!)
1000 future $\approx$ 100-500 KB
Switch cost: ~10-50 ns

Cẩn thận với Future kích thước lớn

```

async fn problematic() {
    let big: [u8; 1_000_000] = [0; 1_000_000]; // 1 MB trong state
    machine!
    some_io().await;
    println!("{}", big[0]);
}

```

→ Tránh giữ buffer to qua **.await**. Drop trước, hoặc dùng **Vec** (chỉ 24 byte trong state machine).

Phần VII: Lock-Free Programming

Triết lý

BLOCKING (Mutex)

Thread giữ lock bị preempt → tất cả ĐÓNG BĂNG.

LOCK-FREE

LUÔN có ít nhất 1 thread tiến triển.
"system-wide progress guaranteed".

WAIT-FREE (cao nhất)

MỌI thread đều hoàn thành sau số bước hữu hạn.

Treiber Stack — Lock-free stack

R. Kent Treiber, 1986.

```
struct Node<T> {
    value: T,
    next: *mut Node<T>,
}

struct TreiberStack<T> {
    head: AtomicPtr<Node<T>>,
}
```

Push

```
fn push(&self, value: T) {
    let new_node = Box::into_raw(Box::new(Node { value, next:
ptr::null_mut() }));
    loop {
        let head = self.head.load(Ordering::Acquire);
        unsafe { (*new_node).next = head; }
        if self.head.compare_exchange(head, new_node, Release,
Acquire).is_ok() {
            return;
        }
    }
}
```

Pop (có vấn đề ABA và use-after-free)

```
fn pop(&self) -> Option<T> {
    loop {
        let head = self.head.load(Ordering::Acquire);
        if head.is_null() { return None; }
        let next = unsafe { (*head).next };
        if self.head.compare_exchange(head, next, Release, Acquire).is_ok()
        {
            let value = unsafe { ptr::read(&(*head).value) };
            unsafe { drop(Box::from_raw(head)); } // 🔥 nguy hiểm!
            return Some(value);
        }
    }
}
```

ABA Problem

ABA: Giá trị thay đổi A → B → A. CAS chỉ thấy "vẫn là A" → tưởng không đổi → nhưng đã có thay đổi ẩn.

Thread T1	Thread T2
1. load head = ptr_A	
2. read A.next = ptr_B	
	3. pop A → free(ptr_A)
	4. pop B → free(ptr_B)
	5. push X → allocator REUSE ptr_A X.next = ptr_C
6. CAS(head, ptr_A, ptr_B) → SUCCESS! (sai!)	
7. head = ptr_B (memory đã free!)	

Giải pháp:

1. **Tagged Pointer (DCAS):** thêm counter, cần CMPXCHG16B.
2. **Hazard Pointers.**
3. **Epoch-based reclamation.**
4. **GC** (Java tránh "miễn phí").

Michael-Scott Queue

Maged Michael & Michael Scott, 1996.

```
struct Node<T> {
    value: Option<T>,
    next: AtomicPtr<Node<T>>,
}

struct MSQueue<T> {
```

```
    head: AtomicPtr<Node<T>>,
    tail: AtomicPtr<Node<T>>,
}
```

Dùng **dummy node** (sentinel) để head và tail không bao giờ null cùng lúc.



Khái niệm "Helping"

Thread A có thể **giúp** Thread B hoàn thành công việc dở dang → đảm bảo system-wide progress.

Memory Reclamation

Phương pháp	Đặc điểm
1. Reference Count	Atomic counter cho mỗi node Đặt mỗi truy cập
2. Hazard Pointers	Mỗi thread đăng ký "đang dùng X" Bounded memory
3. Epoch-based (EBR)	Chia thời gian thành "epoch" Batch free hiệu quả
4. RCU (Linux)	Đọc rẻ, update tạo copy

Hazard Pointers (Maged Michael, 2004)

Mỗi thread có 1 **slot global** công bố: "Tôi đang đọc pointer X."

GLOBAL HAZARD ARRAY:

Thread 0 ptr=0xA00	Thread 1 ptr=null	Thread 2 ptr=0xB10	Thread 3 ptr=0xA00
-----------------------	----------------------	-----------------------	-----------------------

Free P:

- 1. Đặt P vào **retired list**.
- 2. Khi list đầy → quét hazard array.
- 3. P không trong array → **free thật**.

Epoch-Based Reclamation (Crossbeam)

Chia thời gian thành **epoch**. Object chỉ free khi MỌI thread đã rời epoch của nó.

```
use crossbeam_epoch::{self as epoch, Atomic, Owned};

fn pop(stack: &Stack<i32>) -> Option<i32> {
    let guard = epoch::pin(); // ← Vào critical section
    loop {
        let head = stack.head.load(Acquire, &guard);
        match unsafe { head.as_ref() } {
            None => return None,
            Some(node) => {
                let next = node.next.load(Acquire, &guard);
                if stack.head.compare_exchange(head, next, Release,
Acquire, &guard).is_ok() {
                    unsafe { guard.defer_destroy(head); } // schedule free
                    return Some(node.value);
                }
            }
        }
    }
    // guard drop → "rời critical section"
}
```

```
GLOBAL EPOCH = 5
Thread 0  LocalEpoch = 5
Thread 1  LocalEpoch = 5
Thread 2  LocalEpoch = ∅ (không trong guard)
Thread 3  LocalEpoch = 5

GARBAGE:
epoch 5: [obj_A, obj_B]
epoch 4: [obj_C]
epoch 3: [obj_D] ← FREE được nếu mọi thread ≥ epoch 5
```

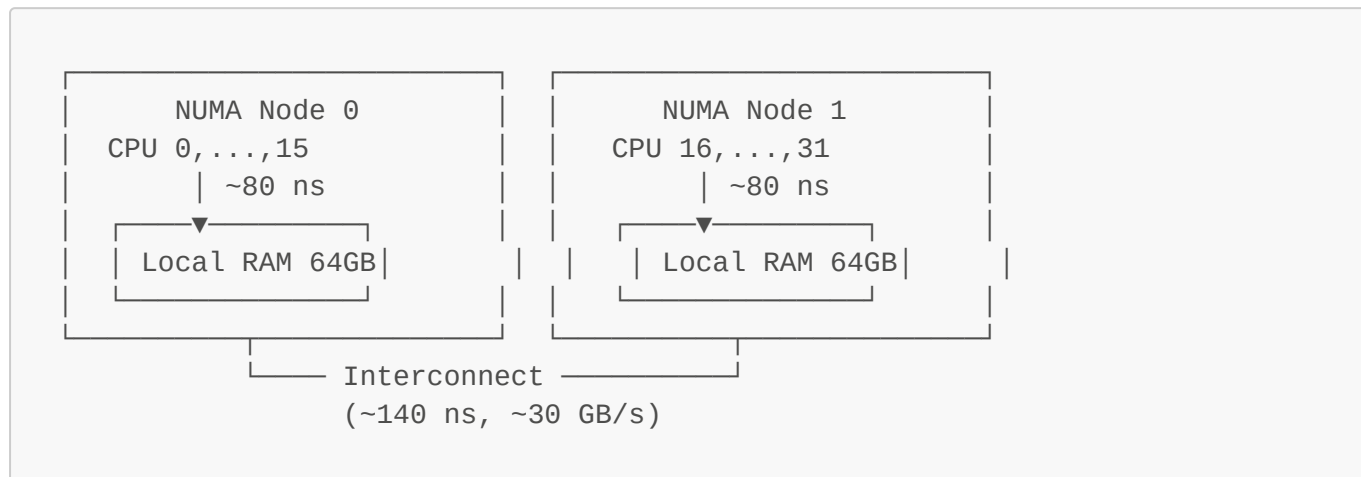
So sánh

EPOCH-BASED	HAZARD POINTERS
Read: rất nhanh	Read: chậm (store + fence)
Free: batch hiệu quả	Free: cá nhân, scan
Memory: có thể grow lớn	Memory: bounded
Phù hợp: tải đọc nhiều	Phù hợp: real-time, bounded mem

Phần VIII: NUMA, Cache Associativity & Prefetching

NUMA — Non-Uniform Memory Access

Trên server hiện đại, RAM **không đồng nhất**:



- CPU 0 → RAM Node 0: 80 ns.
- CPU 0 → RAM Node 1: 140 ns (chậm 1.7x).

NUMA-aware programming

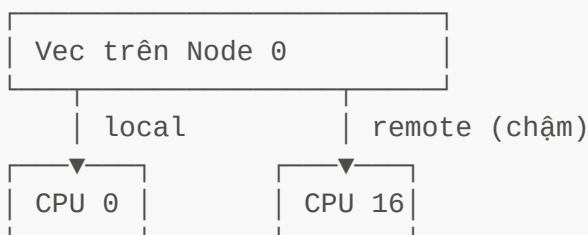
```
// Pin thread vào CPU cụ thể
core_affinity::set_for_current(core_affinity::CoreId { id: 0 });
```

```
# Cấp phát trên node cụ thể
numactl --cpunodebind=0 --membind=0 ./my_program

# Kiểm tra
numactl --hardware
```

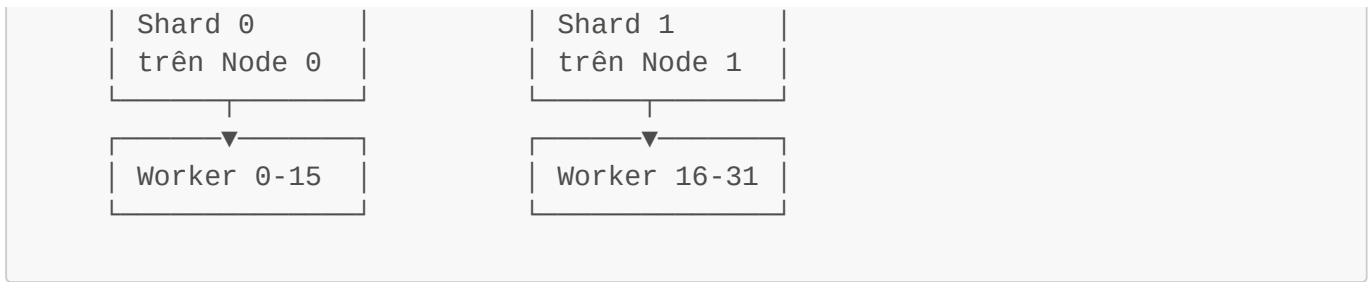
Chiến lược NUMA

❌ BAD: Global Vec



✅ GOOD: Shard per-node





Cache Associativity

Direct-Mapped

Mỗi địa chỉ RAM map vào DUY NHẤT 1 ô cache. → Conflict (thrashing) cao.

Fully Associative

Bất kỳ địa chỉ nào → ô bất kỳ. Đắt → chỉ cache nhỏ.

Set-Associative (N-way)

Cache chia thành **set**, mỗi set có N ô.

L1 8-way (Intel): 32KB, cache line 64B

- Số set = $32\text{KB} / (64\text{B} \times 8) = 64$ sets
- $\text{set_index} = (\text{addr} / 64) \% 64$

Cạm bẫy: Power-of-2 strided access

```

let arr: [[u8; 4096]; 1024];

for i in 0..1024 {
    sum += arr[i][0]; // stride 4096
}
  
```

→ Tất cả truy cập map vào CÙNG SET → thrashing → chậm 10-20x.

Giải pháp:

```

struct PaddedRow {
    data: [u8; 4096],
    _pad: [u8; 64], // tránh power-of-2
}
  
```

Prefetching

Hardware Prefetcher

CPU đoán access tiếp theo và load trước:

- L1 Streamer: pattern tuần tự.
- L1 IP Prefetcher: theo instruction pointer.
- L2 Spatial: load cặp cache line.
- L2 Streamer: pattern tuần tự L2.

Khi hardware thất bại

```
// Linked list – KHÔNG tuần tự
while let Some(node) = current {
    process(node.value);
    current = node.next; // địa chỉ không đoán được
}
```

→ Mỗi `next` cache miss → LinkedList chậm hơn Vec rất nhiều.

Software prefetch

```
use std::intrinsics::prefetch_read_data;

for i in 0..arr.len() {
    if i + 10 < arr.len() {
        unsafe { prefetch_read_data(&arr[i + 10], 3); }
    }
    process(arr[i]);
}
```

Non-temporal store

```
// Bỏ qua cache, ghi thẳng RAM
unsafe { non_temporal_store(&mut arr[i], value); }
```

Khi dùng: ghi tuần tự lượng lớn không đọc lại sớm (memcpy lớn).

Khi nào dùng gì?

Tình huống	Giải pháp
Counter đơn giản	AtomicU64 + Relaxed
Producer-consumer	crossbeam::channel
Map đa luồng	dashmap (sharded)
Stack/Queue lock-free	crossbeam::deque
Tối đa throughput	Lock-free tự viết + epoch
Real-time, bounded latency	Wait-free + hazard pointers
Server đa-socket	NUMA-aware sharding

Hot loop, data-intensive	Software prefetch + SIMD
Cache thrashing nghi ngờ	Đo bằng perf, thêm padding

Công cụ đo lường

```
# Cache miss rate
perf stat -e cache-misses,cache-references ./your_program

# L1 cache miss
perf stat -e L1-dcache-load-misses,L1-dcache-loads ./your_program

# NUMA cross-node
numastat -p <pid>

# Branch mispredict
perf stat -e branch-misses,branches ./your_program

# Full profile
perf record -g ./your_program
perf report
```

Rust crates:

```
[dependencies]
criterion = "0.5"      # benchmark
crossbeam = "0.8"      # lock-free primitives
flame = "0.2"          # flame graph
core_affinity = "0.8"  # CPU pinning
```

Tài liệu tham khảo

Sách

1. **"The Art of Multiprocessor Programming"** — Herlihy & Shavit (kinh thánh lock-free).
2. **"Rust Atomics and Locks"** — Mara Bos (free online: <https://marabos.nl/atomics/>).
3. **"What Every Programmer Should Know About Memory"** — Ulrich Drepper.
4. **"Computer Systems: A Programmer's Perspective"** — Bryant & O'Hallaron.

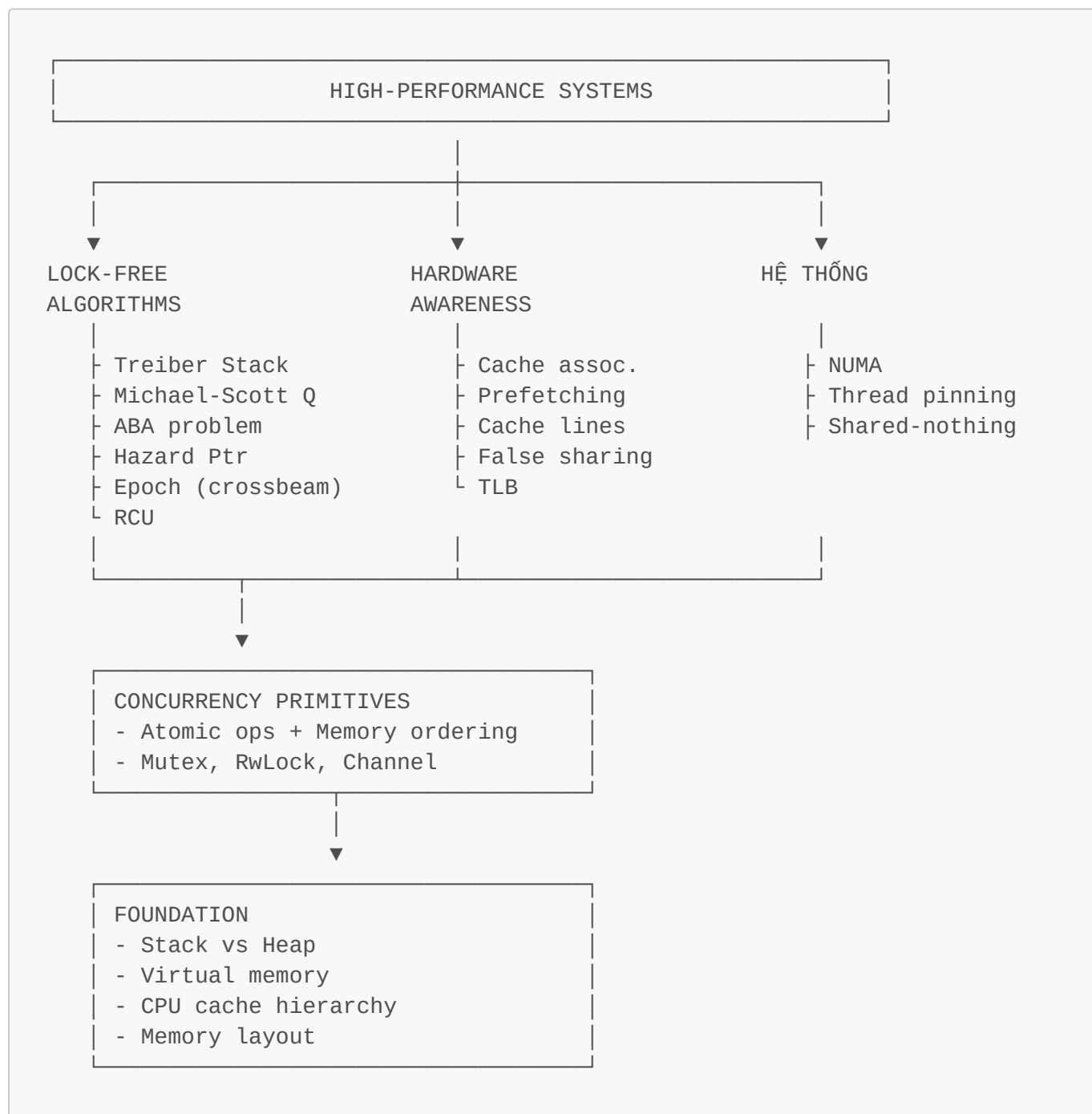
Blog & Online

- Mechanical Sympathy blog — Martin Thompson.
- Preshing on Programming — Jeff Preshing.
- Rust Nomicon: <https://doc.rust-lang.org/nomicon/>.

Source code đáng đọc

- [crossbeam-epoch](#) — Rust epoch-based reclamation.
- [tokio](#) — async runtime.
- [parking_lot](#) — efficient mutex.
- [dashmap](#) — concurrent hashmap.

Bản đồ kết nối tổng quát



Tài liệu này tổng hợp kiến thức memory model từ cơ bản đến nâng cao, dành cho lập trình viên Rust muốn hiểu sâu bản chất của ngôn ngữ. Mọi chủ đề đều có thể đào sâu hơn — đây chỉ là điểm khởi đầu.